

A AURORA ESTABELECE OS
PRIMEIROS SISTEMAS DA COLÔNIA

FACTORES E
CONDIÇÕES QUE
DETERMINAM O
COMPORTAMENTO
INICIAL DA
INFRAESTRUTURA

20,1010

070

5

LISTA DE FIGURAS

Figura 1- Componentes de um vetor com duas dimensões	7
Figura 2 - Sinal de eletrocardiograma em um intervalo de tempo	11
Figura 3 - Organização de um <i>array</i> 2D em linhas e colunas	12
Figura 4 - Ilustração de diferentes tipos de matrizes	16
Figura 5 - Diferentes canais de um equipamento de EEG	19
Figura 6 – Imagem em tons de cinza, com valores dos pixels mostrados na região em destaque.....	20
Figura 7 - Gráfico que representa uma função	22
Figura 8 - Dígitos manuscritos digitalizados do <i>Dataset Digits</i>	23
Figura 9 - Imagens do <i>Dataset Digits</i> empilhadas formando um <i>array</i> 3D	24
Figura 10 - Desempenho na multiplicação de matrizes com <i>loops</i> e com NumPy	25
Figura 11 - Desempenho da operação x^2 com <i>loops</i> e com vetorização.....	26
Figura 12 - Gráfico 3D da função $z = \text{np.sin}(x) + \text{np.cos}(y)$	27
Figura 13 - Comparação de tempo da multiplicação $A @ x$ para matrizes cheias e esparsas.....	28
Figura 14 - Comparação do uso da memória de matrizes cheias e esparsas.....	29
Figura 15 - Sistema de equações sem solução (esquerda) e com solução única (direita)	30
Figura 16 - Representação matricial de um sistema linear	30
Figura 17 - Circuito elétrico que ilustra o problema	31
Figura 18 - Imagem original (esquerda) e bordas destacadas (direita)	33
Figura 19 - Resultado dos ajustes linear e polinomial	35
Figura 20 - Ilustração de duas componentes principais de um conjunto de dados ...	37
Figura 21 - Visualização das duas primeiras componentes principais no <i>wine dataset</i> , com e sem <i>standardization</i> dos dados	38

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Verificação de axiomas do espaço vetorial	10
Código-fonte 2 – Comparação entre <i>arrays</i> 1D e 2D	13
Código-fonte 3 – Operações com <i>arrays</i> 2D	14
Código-fonte 4 – Tipos de matrizes.....	15
Código-fonte 5 – Exemplos de atributos de <i>ndarrays</i>	17
Código-fonte 6 – Exemplos de métodos de <i>ndarrays</i>	18
Código-fonte 7 – Geração de gráfico 3D utilizando <i>broadcasting</i>	27
Código-fonte 8 – Solução de um sistema de equações que representa um circuito elétrico.....	32
Código-fonte 9 – Definição de filtro e aplicação de convolução 2D para detecção de bordas	33
Código-fonte 10 – Ajustes linear e polinomial	34
Código-fonte 11 – PCA do <i>wine dataset</i>	38

SUMÁRIO

MATRIZES E VETORES EM SIMULAÇÕES	5
1 INTRODUÇÃO: A PONTE ENTRE MATEMÁTICA E COMPUTAÇÃO	5
2 VETORES E ESCALARES	7
2.1 Espaços vetoriais	8
2.2 Considerações sobre implementação computacional de arrays	10
2.3 Exemplos de <code>arrays 1D</code>	10
3 MATRIZES E TENSORES	12
3.1 Operações com matrizes	13
3.2 Tipos de matrizes	14
3.3 Classe <code>ndarray</code> : Atributos e métodos	16
3.4 Exemplo de <code>array 2D</code> : <i>Dataset</i> de eletroencefalogramas	19
3.5 Exemplo de <code>array 2D</code> : Imagens em tons de cinza	20
3.6 Tensores, <code>arrays 3D</code> e exemplos	21
4 TÓPICOS SOBRE EFICIÊNCIA COMPUTACIONAL	24
4.1 Evitando <i>loops</i> com rotinas otimizadas	25
4.2 Evitando <i>loops</i> com vetorização	25
4.3 <i>Broadcasting</i>	26
4.4 Aproveitando a estrutura de matrizes esparsas	28
5 APLICAÇÕES	29
5.1 Sistemas de equações lineares	29
5.1.1 Exemplo em circuitos elétricos	31
5.2 Processamento de imagens	32
5.3 Ajuste de curvas	33
5.4 PCA para redução de dimensionalidade	35
5.4.1 Exemplo de uso de PCA em classificação	37
6 CONCLUSÃO	38
REFERÊNCIAS	40
GLOSSÁRIO	41

MATRIZES E VETORES EM SIMULAÇÕES

1 INTRODUÇÃO: A PONTE ENTRE MATEMÁTICA E COMPUTAÇÃO

Nas ciências, compreender fenômenos do mundo depende da nossa capacidade de representá-los. Precisamos traduzir as nossas observações em símbolos, números e operações que capturem suas propriedades essenciais. Sem uma organização e sistematização, seria impossível identificar padrões, comparar situações diferentes ou analisar sistemas complexos. É na necessidade de armazenar, organizar e estruturar dados e processos que os objetos matemáticos se estabelecem, operando sobre informações de maneira consistente e eficiente.

Em Python, a manipulação de dados numéricos na forma de vetores e matrizes é feita de forma muito mais eficiente quando utilizamos bibliotecas dedicadas. Um exemplo é a `NumPy`, projetada para cálculos vetorizados e operações de álgebra linear de alta performance. Tudo começa com o `ndarray`, uma classe orientada a objetos, capaz de armazenar atributos que descrevem seu estado, como dimensões, forma, tipo de dado e informações de memória, e de executar métodos que realizam operações matemáticas, cópias, reorganizações de forma e outras transformações.

Anteriormente vimos como as funções matemáticas podem representar diferentes problemas, além de gerarmos gráficos de diversas funções utilizando Python, `NumPy` e `Matplotlib`. Neste capítulo, vamos descrever e detalhar vários aspectos desse processo que são características eficientes do `NumPy`. Por isso, ao transformar expressões matemáticas em objetos computacionais, devemos compreender como esses valores passam a existir dentro do Python, sua estrutura e como eles são organizados.

Nos gráficos anteriores, definimos intervalos regulares de valores ao longo do eixo x , utilizando funções como `numpy.linspace` ou `numpy.arange`. Em seguida, aplicamos expressões matemáticas, como `numpy.exp` ou `numpy.cos`, sobre esses valores para obter a saída da função. Com x e y disponíveis, gráficos de linhas foram construídos. Podemos detalhar que tanto x quanto y foram armazenados em uma grade unidimensional (ou uma linha) de valores numéricos. Matematicamente,

poderiam ser entendidos como vetores; computacionalmente, eles são identificados como `arrays 1D`, estruturas lineares do NumPy capazes de guardar sequências ordenadas de números.

Essa não é a única maneira de estruturar dados, afinal, nem toda situação pode ser representada apenas por uma sequência única e linear de valores. Ampliando essa ideia, dados também podem ser organizados em linhas e colunas, uma grade bidimensional, sendo as planilhas e matrizes formas possíveis.

Uma planilha é uma forma bastante intuitiva de trabalhar com dados tabulares. Nelas, colunas diferentes podem armazenar tipos diferentes de variáveis (categóricas ou numéricas), e o usuário pode facilmente visualizar, editar e aplicar fórmulas. No Python, é possível representá-las como `DataFrames` do Pandas, que adicionam rótulos para linhas e colunas e permitem manipulação de dados heterogêneos para realizar análises estruturadas.

Apesar dessa flexibilidade, as planilhas não possuem uma estrutura matemática dedicada para operações algébricas. Como alternativa, a matriz é um objeto matemático também organizado em linhas e colunas, em que todos os seus elementos são do mesmo tipo (numéricos). Isso permite realizar operações como soma, multiplicação ou transposição de forma otimizada. No Python, é possível criá-las como `arrays 2D` do NumPy.

Vemos que a escolha da estrutura adequada depende do tipo de informação que desejamos manipular e das operações que pretendemos realizar. A organização de informações em grades `1D`, `2D` ou com maior dimensionalidade é a estratégia fundamental para manipular grandes volumes de dados de forma eficiente.

Assim, esse material tem como objetivo apresentar os conceitos matemáticos da álgebra linear, como vetores, matrizes e tensores. Eles serão vistos computacionalmente, com ênfase nos `arrays n-dimensionais` do NumPy, para resolver problemas que seriam praticamente impossíveis de tratar manualmente.

2 VETORES E ESCALARES

As primeiras estruturas matemáticas que vamos estudar para descrever fenômenos são os escalares e os vetores.

Os escalares representam valores numéricos isolados, como constantes ou parâmetros. Eles descrevem, por exemplo, massa, temperatura, tempo, comprimento ou volume, grandezas que só possuem magnitude (ou tamanho).

Já os vetores, em uma definição inicial e intuitiva, representam grandezas que possuem tamanho, direção e sentido. Exemplos incluem grandezas físicas como forças, posições, velocidades ou acelerações. Esse tipo de vetor é conhecido como vetor geométrico ou euclidiano. Conforme demonstra a figura a seguir, é usual representá-los com setas e o resultado é um agrupamento de suas componentes. Para o caso de duas dimensões, temos um vetor $\mathbf{v}$ com duas componentes (v_x , v_y).

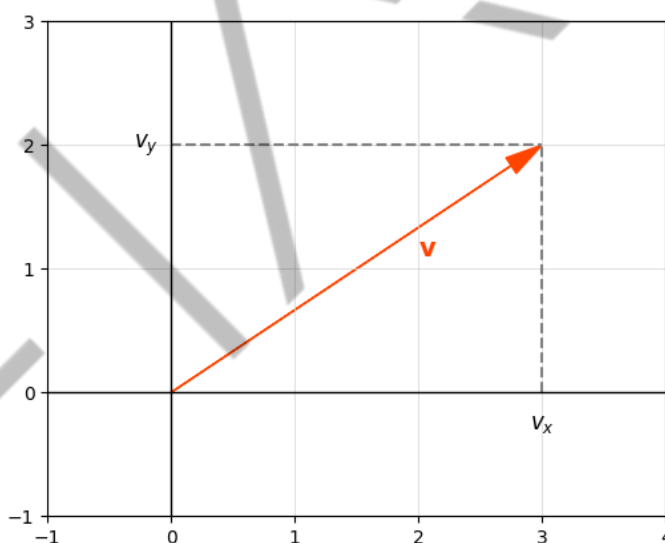


Figura 1- Componentes de um vetor com duas dimensões
Fonte: Elaborada pelo autor (2025)

Observação: a notação de escalares costuma ser uma letra minúscula, grega ou não. Já a notação de vetores pode mudar conforme a referência: pode ser uma letra com uma seta por cima ou uma letra minúscula em negrito, que é a forma que seguiremos nesse capítulo.

Cada componente do vetor indica quanto ele avança em cada direção do espaço; nesse caso ao longo dos eixos x e y . A combinação dessas componentes determina o comprimento do vetor e a direção na qual ele aponta. O tamanho do vetor, chamado magnitude, é calculado aplicando o teorema de Pitágoras com as suas

componentes. Em duas dimensões: $|\mathbf{v}| = (v_x^2 + v_y^2)^{1/2}$; em três dimensões: $|\mathbf{v}| = (v_x^2 + v_y^2 + v_z^2)^{1/2}$, e assim por diante. Esse comprimento corresponde à distância entre a origem do espaço e a “ponta” do vetor. A direção e o sentido do vetor decorrem das proporções entre suas componentes: vetores como (2,2) e (4,4) apontam na mesma direção, embora tenham tamanhos diferentes.

Observação: quando indicamos componentes específicos de um vetor, escrevemos sem negrito e costuma-se acompanhar um subscrito indicando o eixo, índice ou outra informação relevante.

Para determinar o ângulo entre dois vetores, utilizamos o produto escalar, definido por $\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}| \cos(\theta)$. Ele mede o quanto um vetor aponta na direção do outro, permitindo calcular o ângulo entre os vetores. Para isso, podemos isolar $\theta = \arccos[(\mathbf{a} \cdot \mathbf{b})/(|\mathbf{a}||\mathbf{b}|)]$, válida para qualquer tamanho de vetor.

Na física, a distinção entre vetores e escalares é essencial. Um exemplo é a segunda lei de Newton, $\mathbf{F} = m\mathbf{a}$. A massa m é um escalar: não aponta para lugar nenhum, é apenas um valor. A aceleração $\mathbf{a}$ é um vetor, pois envolve mudança de velocidade em uma direção específica. Logo, a força $\mathbf{F}$ resultante também é um vetor, justamente porque ela é proporcional à aceleração. A massa funciona apenas como um fator que estica ou reduz o tamanho do vetor, sem alterar sua direção. Isso explica por que forças não podem ser somadas como números comuns: é preciso levar em conta suas componentes e a direção em que atuam.

2.1 Espaços vetoriais

Quando começamos a estudar vetores, a interpretação geométrica de setas com tamanho, direção e sentido é extremamente útil para criar intuição, mas é apenas um caso particular. A ideia fundamental de vetor é a de um objeto matemático que pode ser somado a outros e multiplicado por números, obedecendo certas regras. Nesse sentido, essa especificidade abre espaço para representar muitos outros tipos de informação.

Em uma definição mais geral, vetores são elementos de espaços vetoriais. Um espaço vetorial V é um conjunto que reúne elementos que obedecem a certos axiomas de soma e multiplicação por escalar, mostrados a seguir:

- Comutatividade da adição - Quando temos dois vetores, não importa a ordem da soma deles: para todos $\mathbf{u}, \mathbf{v}$ em V , vale $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$
- Associatividade da adição - Quando temos três vetores, não importa como agrupamos eles para somarmos: $\mathbf{u} + (\mathbf{v} + \mathbf{w}) = (\mathbf{u} + \mathbf{v}) + \mathbf{w}$
- Existência do elemento neutro aditivo - Existe um vetor que, quando somado com qualquer outro, não altera o resultado: existe $\mathbf{0}$ em V tal que $\mathbf{u} + \mathbf{0} = \mathbf{u}$
- Existência do inverso aditivo - Cada vetor precisa ter um inverso aditivo, ou seja, um “oposto” que, quando somado ao original, resulta no vetor nulo: para cada $\mathbf{u}$ em V , existe $-\mathbf{u}$ em V tal que $\mathbf{u} + (-\mathbf{u}) = \mathbf{0}$
- Associatividade da multiplicação por escalar - Multiplicar o vetor por b e depois multiplicar por a é igual a multiplicar ab de uma vez: $a(b\mathbf{u}) = (ab)\mathbf{u}$
- Distributividade do escalar sobre a adição de vetores - Um escalar se distribui sobre a soma de vetores: $a(\mathbf{u} + \mathbf{v}) = a\mathbf{u} + a\mathbf{v}$
- Distributividade da multiplicação sobre a adição de escalares - A multiplicação de um vetor se distribui sobre a soma de escalares: $(a + b)\mathbf{u} = a\mathbf{u} + b\mathbf{u}$
- Identidade multiplicativa do escalar - Multiplicar por 1 não muda as características do vetor: $1 \cdot \mathbf{u} = \mathbf{u}$

Essas propriedades podem parecer abstratas ou mesmo óbvias, mas elas são importantes para definir como as operações são realizadas com vetores. Para ilustrar, o código-fonte a seguir verifica os axiomas com `arrays 1D` do NumPy. `Ndarrays` não foram criados para “serem um espaço vetorial”, mas acabam obedecendo exatamente às mesmas regras, ilustrando-as.

```
import numpy as np
u = np.array([1.0, 2.0, 3.0])
v = np.array([4.0, 5.0, 6.0])
w = np.array([-1.0, 0.5, 2.0])
a = 3.0
b = -2.0
# comutatividade
print("u + v =", u + v)
print("v + u =", v + u)
# associatividade da adição
print("u + (v + w) =", u + (v + w))
```

```
print("(u + v) + w =", (u + v) + w)
# elemento neutro
zero = np.zeros_like(u)
print("u + zero =", u + zero)
# inverso aditivo
print("u + (-u) =", u + (-u))
# identidade multiplicativa (1 * u = u)
print("1 * u =", 1 * u)
# associatividade da multiplicação escalar
print("a * (b * u) =", a * (b * u))
print("(a * b) * u =", (a * b) * u)
# distributividade do escalar na soma de vetores
print("a * (u + v) =", a * (u + v))
print("a*u + a*v =", a*u + a*v)
# distributividade na soma de escalares
print("(a + b) * u =", (a + b) * u)
print("a*u + b*u =", a*u + b*u)
```

Código-fonte 1 – Verificação de axiomas do espaço vetorial
Fonte: Elaborado pelo autor (2025)

2.2 Considerações sobre implementação computacional de arrays

É importante notar que existem analogias entre objetos matemáticos com tipos de variáveis numéricas, por exemplo: as vezes encontramos que escalares são `arrays 0D` e vetores são `arrays 1D`. No entanto, é necessário cuidado, pois não são equivalências totais; na documentação do `NumPy`, isso é detalhado.

De qualquer forma, queremos representá-los computacionalmente. O papel do escalar, como quantidade que pode multiplicar outras, é desempenhado no `NumPy` por tipos como `int`, `float` ou versões mais precisas como `np.float64`. Um vetor pode ser entendido como uma lista ordenada de números que respeita determinadas propriedades matemáticas. Um `array 1D` representa essa ideia: uma sequência de números organizada de forma consistente, em que cada elemento possui um índice que permite acesso direto. Vale destacar que um vetor geométrico de `n` dimensões pode ser representado como um `array 1D` com `n` elementos.

2.3 Exemplos de arrays 1D

Podemos representar muitos elementos diferentes com `arrays 1D`:

- Vetores geométricos em `n` dimensões;

- Vetor de parâmetros de um modelo;
- Séries temporais discretas, como áudios ou sinais fisiológicos;
- Vetor de características (*features*) em *machine learning*.

Detalharemos um exemplo específico de um sinal biomédico, chamado de eletrocardiograma (ECG). Ele é uma série temporal contendo a variação do potencial elétrico medido no tórax ao longo do tempo, refletindo a atividade elétrica do coração. Há, portanto, um `array` de tempo e outro que representa o valor da amplitude do sinal em um instante específico. De modo geral, ele é considerado um sinal 1D, porque contém apenas uma grandeza variável: a amplitude do sinal ao longo do tempo. A figura “Sinal de eletrocardiograma em um intervalo de tempo” mostra um exemplo.

Embora o sinal seja unidimensional, sua representação gráfica utiliza dois eixos. Isso ocorre porque o processo de visualização exige que cada amostra do sinal seja associada a um valor no eixo horizontal. Assim, ao representar o sinal graficamente, criamos explicitamente uma escala temporal (eixo x), enquanto o valor do sinal é colocado no eixo vertical (eixo y). No gráfico resultante, é possível identificar componentes típicos do ECG que refletem diferentes fases do ciclo cardíaco, além de possíveis irregularidades ou artefatos no sinal.

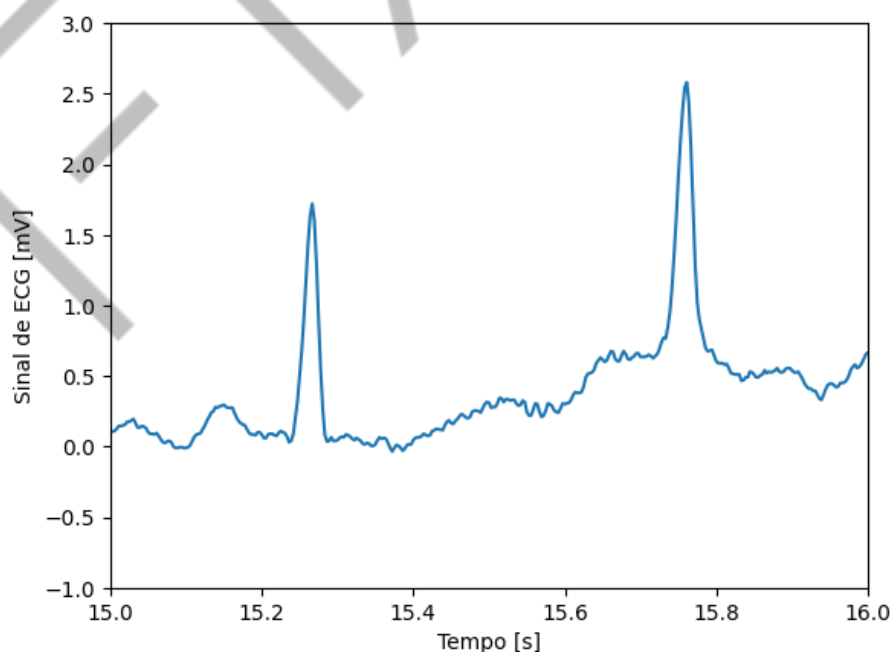


Figura 2 - Sinal de eletrocardiograma em um intervalo de tempo
Fonte: Elaborada pelo autor (2025)

3 MATRIZES E TENSORES

Além dos vetores, que são estruturas unidimensionais, podemos construir matrizes ao organizar os elementos em linhas e colunas, criando assim uma estrutura bidimensional. Essa ideia de ampliar dimensões permite armazenar conjuntos maiores de dados de forma organizada. Por exemplo, considere uma matriz com m linhas e n colunas. Cada elemento é um número real e a matriz em si pertence ao conjunto $\mathbb{R}^{m \times n}$. O sobrescrito “ $m \times n$ ” indica que a matriz tem duas dimensões estruturais: m linhas e n colunas. Logo, precisamos de mn números reais para especificar qualquer matriz nesse espaço.

Quando partimos para a implementação computacional das matrizes, podemos representá-las como `arrays` bidimensionais. No NumPy, linhas e colunas também são chamadas de *axis* (eixos) do `array` 2D. Isso é mostrado na figura “Organização de um `array` 2D em linhas e colunas”. O código-fonte a seguir compara brevemente `arrays` 1D e 2D.

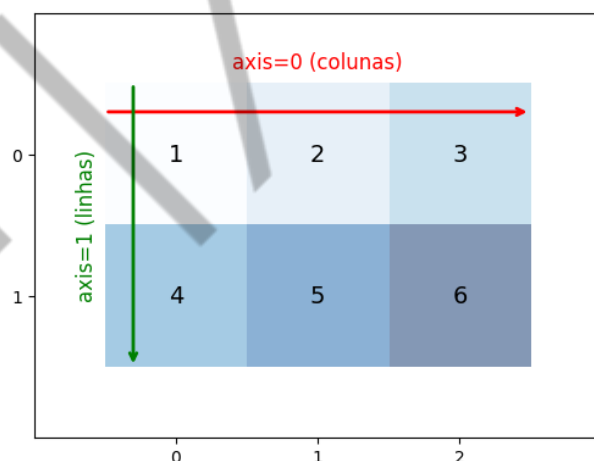


Figura 3 - Organização de um `array` 2D em linhas e colunas
Fonte: Elaborada pelo autor (2025)

```
import numpy as np
x1 = np.array([1, 2, 3]) # Array 1D
print("Transposta x1.T:\n", x1.T) # Transposta em 1D (não muda)
print("Soma total x1.sum() \n:", x1.sum()) # Soma (único eixo)

x2 = np.array([[1, 2, 3], # Array 2D
               [4, 5, 6]])
print("Transposta x2.T:\n", x2.T) # Transposta em 2D
x2.sum(axis=0, keepdims=True) # Soma ao longo do eixo 0 (colunas)
x2.sum(axis=1, keepdims=True) # Soma ao longo do eixo 1 (linhas)
```

Arrays 2D podem representar matrizes matemáticas, tabelas de dados ou agrupamentos de *arrays* 1D (*datasets*), além de serem utilizados em imagens digitais, grafos e suas conexões, transformações lineares em 2D, filtros e *kernels* no processamento de imagens, bem como em sequências temporais com múltiplas variáveis, entre outras aplicações.

3.1 Operações com matrizes

As matrizes permitem diversas operações para manipulação e análise de dados. Nelas, é importante entender como o formato das matrizes envolvidas são compatíveis com as operações que se deseja realizar. Exemplos incluem:

- Soma e a subtração de elementos de duas matrizes em posições correspondentes;
- Multiplicação, que pode ser feita entre uma matriz e um escalar, alterando a escala de todos os elementos;
- Multiplicação entre duas matrizes, combinando linhas e colunas de modo que cada elemento do resultado dependa de múltiplos elementos das matrizes originais;
- Transposição, que troca linhas por colunas;
- Inversão, que busca uma matriz que, quando multiplicada pela original, produz a matriz identidade;
- Determinante, que é um valor associado a uma matriz quadrada que indica, de forma compacta, se a matriz pode ser invertida.

Cada uma das operações citadas tem implementações diretas na biblioteca NumPy, principalmente no módulo `linalg`. O código-fonte “Operações com *arrays* 2D” descreve algumas delas. Isso permite que realizemos um número elevado de operações que não seriam adequadas de outra forma, incluindo multiplicações matriciais e calcular determinantes e matrizes inversas de matrizes muito grandes.

```
import numpy as np
```

```
# Matrizes de exemplo
A = np.array([[2, 1], [5, 3]])
B = np.array([[1, 4], [2, 1]])
# Operações básicas
soma = A + B
sub = A - B
multiplicacao_escalar = 3 * A
multiplicacao_matricial = A @ B # ou np.dot(A, B)
hadamard = A * B # multiplicação elemento a elemento
# Funções de numpy
transposta = np.transpose(A)

det_A = np.linalg.det(A)
A_inv = np.linalg.inv(A)
rank_A = np.linalg.matrix_rank(A)
trace_A = np.trace(A)

# Mostrar resultados
print("Soma:\n", soma)
print("Subtração:\n", sub)
print("Multiplicação por escalar:\n", multiplicacao_escalar)
print("Multiplicação matricial:\n", multiplicacao_matricial)
print("Transposta:\n", transposta)
print("Multiplicação elemento a elemento (Hadamard):\n", hadamard)
print("\nDeterminante:", det_A)
print("Inversa:\n", A_inv)
print("Rank:", rank_A) #nº de linhas linearmente independentes
print("Traço:", trace_A) #Soma dos elementos da diagonal
```

Código-fonte 3 – Operações com arrays 2D
Fonte: Elaborado pelo autor (2025)

3.2 Tipos de matrizes

Dependendo das características dos elementos de uma matriz e da forma como eles estão organizados, podemos classificá-la de diferentes maneiras. Algumas classificações dependem dos valores específicos, como no caso da matriz identidade, que é uma matriz diagonal cujos elementos da diagonal principal são todos iguais a 1.

Outras classificações dependem da estrutura interna, como as matrizes diagonais, que possuem elementos não nulos apenas na diagonal principal, ou as matrizes triangulares, nas quais todos os elementos acima (triangular inferior) ou

abaixo (triangular superior) da diagonal principal são nulos. Há também matrizes simétricas, caracterizadas pela igualdade entre a matriz e sua transposta ($A = A^T$), de modo que os elementos acima da diagonal são espelhados abaixo dela.

Por fim, podemos considerar matrizes densas, que possuem poucos ou nenhum zero, em contraste com matrizes esparsas, nas quais a maioria dos elementos é nula. Assim, algumas classificações dizem respeito ao conteúdo numérico, enquanto outras tratam do padrão estrutural dos elementos dentro da matriz. No código-fonte “Tipos de matrizes” e na figura “Ilustração de diferentes tipos de matrizes” são mostrados alguns desses exemplos.

```
import numpy as np
A_id = np.eye(5) # identidade
A_diag = np.diag([1, 2, 3, 4, 5]) # diagonal
A_sym = np.array([      # simétrica
    [2, -1, 0, 3, 1],
    [-1, 2, 4, -2, 0],
    [0, 4, 1, 1, 2],
    [3, -2, 1, 5, -1],
    [1, 0, 2, -1, 3]])
A_upper = np.triu(np.random.randint(-3, 4, (5, 5))) # triangular
A_full = np.random.randint(-5, 6, (5, 5)) # aleatória e cheia
mask = np.random.rand(5, 5) < 0.2
A_sparse = np.zeros((5, 5))
A_sparse[mask] = np.random.randint(-5, 6, mask.sum()) # esparsa
```

Código-fonte 4 – Tipos de matrizes

Fonte: Elaborado pelo autor (2025)

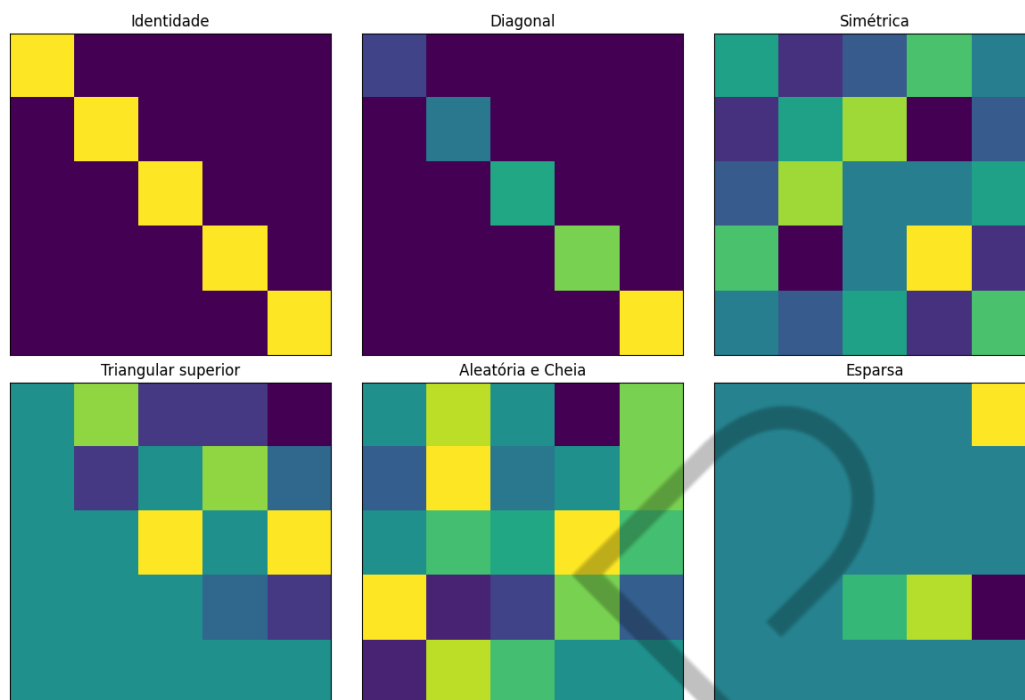


Figura 4 - Ilustração de diferentes tipos de matrizes
Fonte: Elaborada pelo autor (2025)

3.3 Classe `ndarray`: Atributos e métodos

Python é uma linguagem cuja estrutura interna se apoia fortemente na programação orientada a objetos. Isso significa que praticamente tudo é tratado como um objeto. Da mesma forma, um `ndarray` é uma classe definida pelo NumPy (`numpy.ndarray`), e cada `array` criado é uma instância dessa classe, possuindo atributos e métodos próprios.

Os atributos de um `ndarray` são informações armazenadas dentro do objeto que descrevem seu estado, como sua forma, número de dimensões ou tipo dos dados. Eles fornecem características que influenciam como o `array` pode ser manipulado em operações posteriores. Por outro lado, os métodos são funções associadas ao objeto que realizam ações, como transformar a forma do `array`, produzir cópias, aplicar operações ao longo de eixos ou retornar novos objetos derivados.

Nos códigos-fontes “Exemplos de atributos de `ndarrays`” e “Exemplos de métodos de `ndarrays`” são apresentados, respectivamente, exemplos de atributos e métodos associados aos `ndarrays`, ilustrando como esse modelo orientado a objetos organiza tanto os dados quanto os comportamentos disponíveis para cada `array`.

```
import numpy as np
```



```
x = np.arange(12).reshape(3,4) # array 3x4

# Forma e dimensões
print("\nNúmero de dimensões (ndim):", x.ndim)
print("Forma (shape):", x.shape)
print("Número total de elementos (size):", x.size)

# Tipo de dado
print("\nTipo de dado (dtype):", x.dtype)
print("Tamanho de cada elemento (itemsize):", x.itemsize, "bytes")
print("Tamanho total em bytes (nbytes):", x.nbytes, "bytes")

# Estrutura de memória
print("\nStrides (passo em bytes por eixo):", x.strides)
print("Buffer de dados (data):", x.data)
print("Flags do array:\n", x.flags)
print("Array base (base):", x.base) # None se não for view

# Operações matemáticas
print("\nTransposta (T):\n", x.T)
print("Parte real (real):\n", x.real)
print("Parte imaginária (imag):\n", x.imag)
```

Código-fonte 5 – Exemplos de atributos de ndarrays
Fonte: Elaborado pelo autor (2025)

```
import numpy as np
x = np.arange(12).reshape(3,4) # array 3x4

# Cópia e memória
x_copy = x.copy() # cópia independente
x_view = x.view() # view (compartilha memória)
x_flatten = x.flatten() # cópia achatada
x_ravel = x.ravel() # view achatada, se cópia não for necessária
print("\nCópia x_copy:\n", x_copy)
print("View x_view:\n", x_view)
print("Flatten x_flatten:\n", x_flatten)
print("Ravel x_ravel:\n", x_ravel)

# Modificando a view para mostrar efeito
x_view[0,0] = 999
print("\nArray x após modificar x_view[0,0]:\n", x)

# Redimensionamento e dimensões
reshaped = x.reshape(4,3) #mudar formato
transposed = x.T #transposição
swapped = x.swapaxes(0,1) #inverter eixos
squeezed = x.reshape(3,1,4).squeeze() #suprimir um eixo
expanded = x[:, :, np.newaxis] #criar novo eixo
print("\nReshape (4x3):\n", reshaped)
print("Transposta:\n", transposed)
```

```
print("Eixos trocados:\n", swapped)
print("Squeezed (removendo dimensões 1) shape:", squeezed.shape)
print("Expanded (adicionando dimensão extra) shape:", expanded.shape)

# Operações matemáticas e agregação
soma_total = x.sum() #soma de todos os elementos
soma_eixo0 = x.sum(axis=0) #soma em determinada direção
media_eixo1 = x.mean(axis=1) #média em determinada direção
produto_total = x.prod() #produto dos elementos
clipped = x.clip(1,10) # limitar valores a intervalo [1,10]
print("\nSoma total:", soma_total)
print("Soma ao longo do eixo 0:\n", soma_eixo0)
print("Média ao longo do eixo 1:\n", media_eixo1)
print("Soma cumulativa ao longo do eixo 0:\n", soma_cumulativa)
print("Produto total:", produto_total)

# Booleanos e condicionais
algun_maior5 = (x>5).any()
todos_menor20 = (x<20).all()
print("\nAlgun elemento >5:", algun_maior5)
print("Todos os elementos <20:", todos_menor20)
```

Código-fonte 6 – Exemplos de métodos de ndarrays
Fonte: Elaborado pelo autor (2025)

Muitas características de um `ndarray` se aplicam de forma consistente a arrays de qualquer dimensão, 1D, 2D ou N-D. Atributos como `shape`, `ndim`, `size`, `dtype`, `itemsize`, `nbytes`, `strides`, `T`, `flat` e `base` fornecem informações estruturais e funcionam de maneira semelhante independentemente do número de eixos. Da mesma forma, métodos como `copy()`, `reshape()`, `flatten()`, `ravel()`, `sum()`, `mean()`, `any()`, `all()` e `dot()` estão disponíveis em arrays de qualquer dimensão, embora alguns deles apresentem comportamentos específicos conforme a forma do array.

Apesar dessa uniformidade, alguns aspectos exigem atenção especial em arrays 2D ou superiores. A transposta `T`, por exemplo, não altera a estrutura de um array 1D, mas em arrays 2D inverte linhas e colunas e em dimensões maiores reorganiza eixos de acordo com regras mais gerais. Operações que envolvem eixos, como soma ou média, também se tornam mais ricas: em um array 1D existe apenas um eixo possível, enquanto em 2D podemos somar ao longo de linhas ou colunas e em N-D podemos agregar dados em qualquer eixo disponível. Além disso, a

indexação multidimensional permite selecionar linhas, colunas, blocos ou fatias mais complexas, oferecendo um nível de controle que não existia em `arrays 1D`.

3.4 Exemplo de `array 2D`: Dataset de eletroencefalogramas

O *EEG Eye State Dataset* é um conjunto de dados de sinais eletroencefalográficos (EEG) obtidos a partir de um único participante. Cada canal do EEG corresponde a um eletrodo posicionado em uma região específica da cabeça e produz um sinal `1D` ao longo do tempo, representando a variação da atividade elétrica cerebral naquela área.

Há 14 canais que obtiveram sinais registrados simultaneamente. Seu objetivo principal é permitir a análise automática do estado dos olhos (abertos ou fechados) com base na atividade elétrica cerebral em diferentes regiões, associando-as com funções cognitivas, emocionais e sensório-motoras. A figura “Diferentes canais de um equipamento de EEG” mostra alguns exemplos desses sinais.

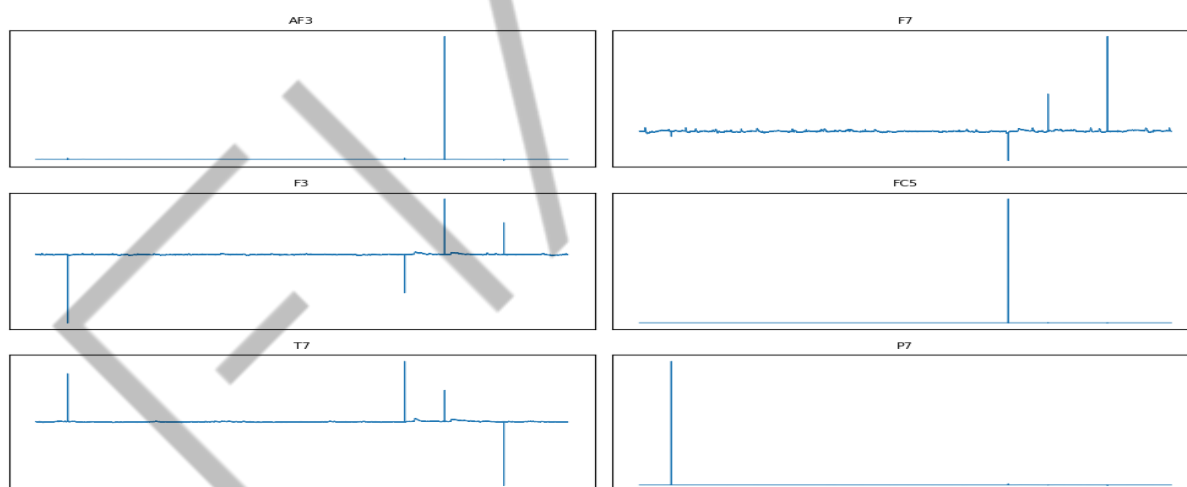


Figura 5 - Diferentes canais de um equipamento de EEG
Fonte: Elaborada pelo autor (2025)

Nesse exemplo, os dados poderiam ser armazenados como um `array 2D`, em que cada linha representa um ponto no tempo e cada coluna corresponde a um canal de EEG. No entanto, podemos também armazená-los em um *DataFrame*, mantendo os nomes das *features* nas colunas.

Nos dois casos, o conjunto como um todo constitui uma série temporal multivariada, na qual múltiplos sinais `1D` coexistem de maneira alinhada no tempo, permitindo análises conjuntas e exploração de padrões temporais complexos.

3.5 Exemplo de *array* 2D: Imagens em tons de cinza

Nesse exemplo, uma imagem em preto e branco é armazenada como um *array* 2D de intensidade. A partir disso, foi destacada uma pequena região em que se mostra o valor numérico de cada pixel dentro desse bloco. A figura “Imagem em tons de cinza, com valores dos pixels mostrados na região em destaque” demonstra a relação entre a imagem e sua representação computacional.

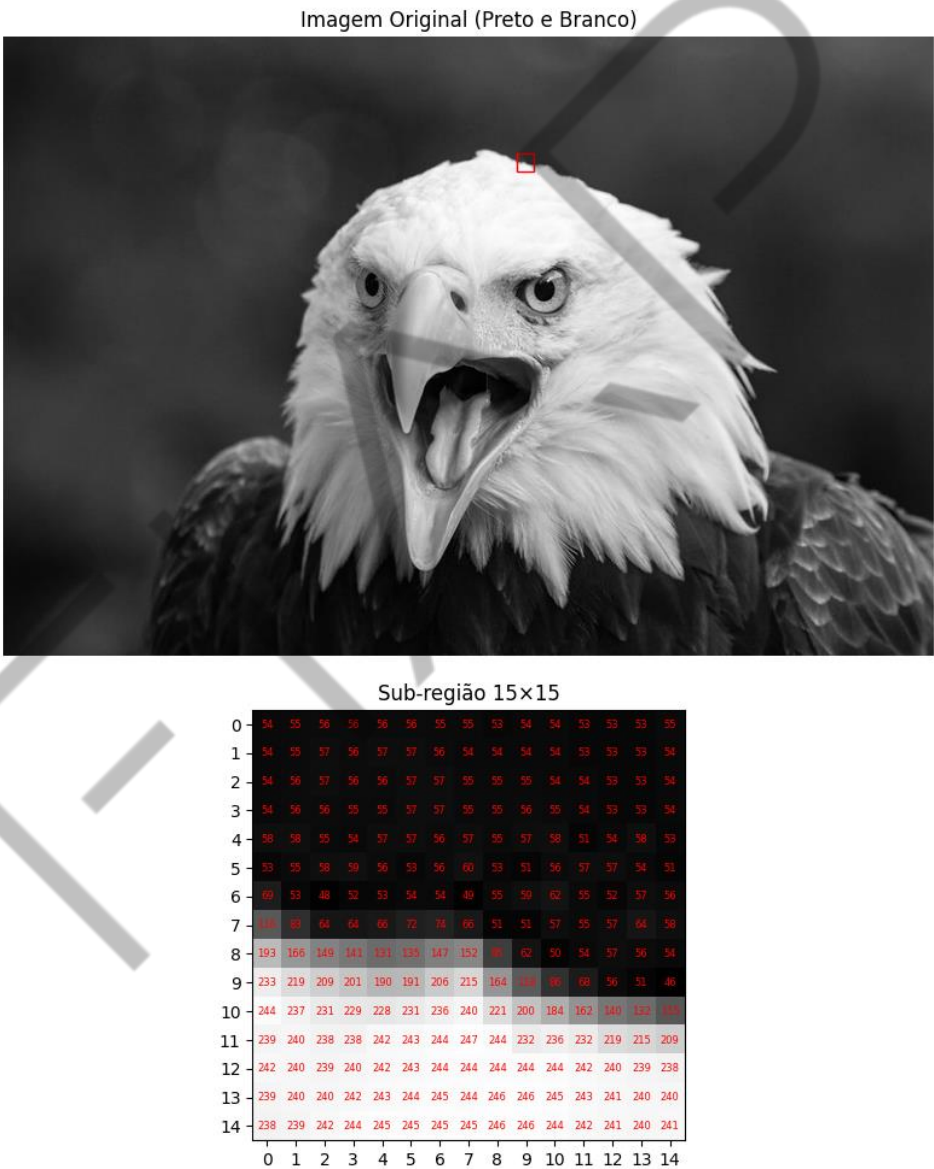


Figura 6 – Imagem em tons de cinza, com valores dos pixels mostrados na região em destaque
Fonte: Elaborada pelo autor (2025)

3.6 Tensores, *arrays* 3D e exemplos

Além de ampliarmos nossa representação de vetores para matrizes, é possível generalizar matrizes para tensores, que são estruturas capazes de ter três, quatro ou mais dimensões. Em sua forma mais simples, um tensor pode ser visto como uma coleção organizada de números distribuídos ao longo de vários eixos: altura, largura, profundidade, canais, tempo e assim por diante. Matematicamente, chamamos de ordem (ou *rank*) a quantidade de dimensões que ele possui: um escalar é um tensor de ordem 0, um vetor é de ordem 1, uma matriz é de ordem 2 e todas as estruturas com mais eixos são consideradas tensores de ordem superior.

Na matemática e na física, o conceito de tensor vai além dessa organização multidimensional. Em termos gerais, ele é um objeto que descreve relações entre quantidades que podem variar em diferentes direções ao mesmo tempo. Além disso, generaliza ideias como “comprimento”, “direção” ou “área”, permitindo representar propriedades que dependem de mais de uma direção ou variável simultaneamente. Exemplos incluem forças que atuam em diversas orientações ou as deformações que ocorrem em um material quando empurrado ou esticado.

Embora essa definição matemática seja mais abstrata, na prática computacional trabalhamos com tensores como `ndarrays` de várias dimensões, o que fornece uma maneira concreta de armazenar e manipular dados complexos. Isso permite representar:

- Imagens coloridas (altura x largura x canais);
- Volumes tridimensionais (altura x largura x profundidade);
- Vídeos (frames x altura x largura x canais);
- Conjuntos inteiros de exemplos usados em aprendizado de máquina (amostra x altura x largura).

Por isso, a computação moderna, especialmente em inteligência artificial e visão computacional, transformou tensores em estruturas fundamentais para o processamento de grandes coleções de dados. A seguir veremos dois exemplos que ilustram essa representação prática.

A figura “Gráfico que representa uma função” mostra uma imagem colorida com um pequeno recorte dentro dela. Nessa região, geramos visualizações dos três canais de cores que formam a imagem: vermelho, verde e azul (R, G e B).

Outro exemplo é quando temos um conjunto de várias imagens 2D: podemos organizá-las em um `array` 3D, como se elas estivessem empilhadas. Para ilustrar, observe o *Dataset Digits* do *scikit-learn*, que contém imagens de dígitos manuscritos.

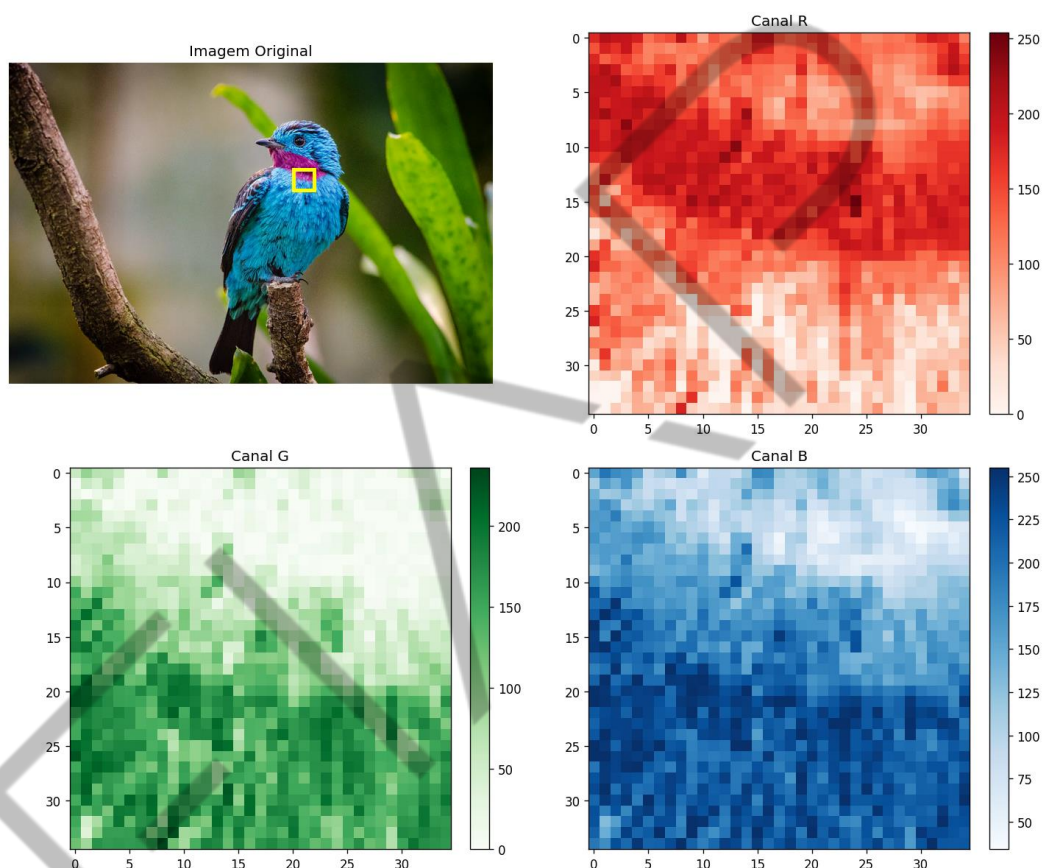


Figura 7 - Gráfico que representa uma função
Fonte: Elaborada pelo autor (2025)

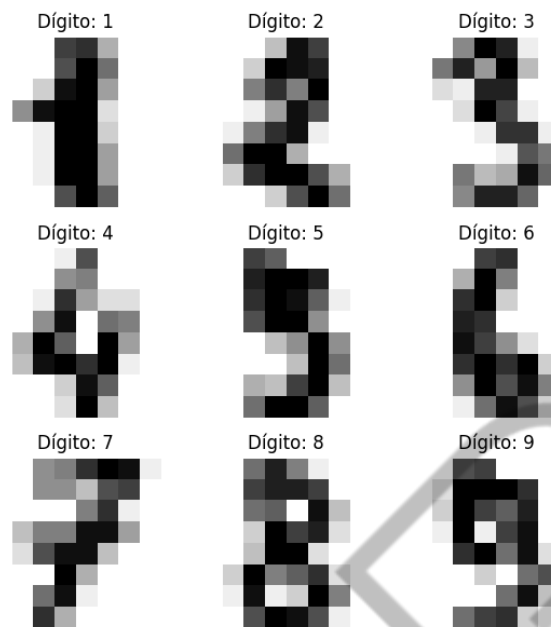


Figura 8 - Dígitos manuscritos digitalizados do *Dataset Digits*
Fonte: Elaborada pelo autor (2025)

No *Dataset Digits*, os dados estão organizados na forma de um `array 3D` com dimensões (amostra, altura, largura). Especificamente, temos um formato de (1797, 8, 8). Isso significa que há 1797 imagens de tamanho 8x8 que podem ser utilizadas para treinar modelos de aprendizado de máquina, por exemplo.

A figura a seguir mostra imagens desse tensor deslocadas ao longo do eixo z para empilhar visualmente as camadas, criando uma percepção de tensor 3D. Mesmo que os dados originais sejam 2D, essa abordagem permite explorar visualmente a ideia de N-dimensões e fatias de um tensor.

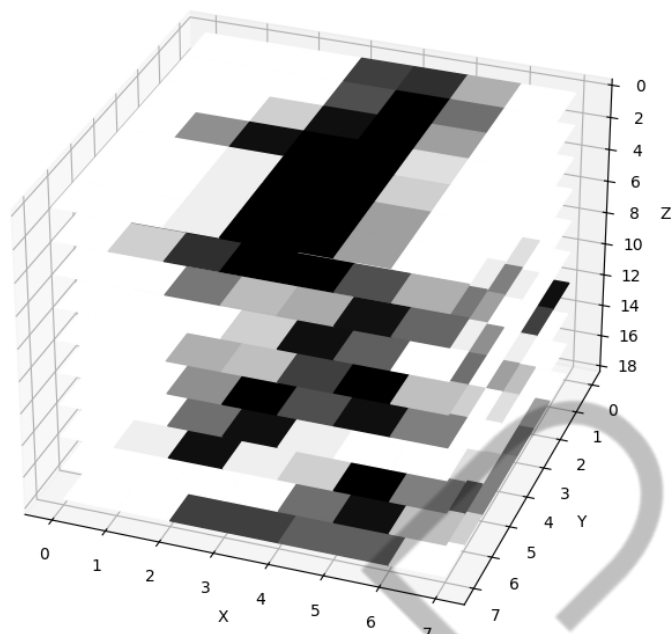


Figura 9 - Imagens do *Dataset Digits* empilhadas formando um *array* 3D
Fonte: Elaborada pelo autor (2025)

4 TÓPICOS SOBRE EFICIÊNCIA COMPUTACIONAL

Na programação científica, escrever código rápido, legível e confiável é essencial. Portanto, estudar a estrutura do `ndarray` é importante para escrever operações matemáticas de modo eficiente e confiável. Isso vai além de aprender sobre uma estrutura específica: é necessário entender os fundamentos de eficiência, manipulação de dados e operações multidimensionais que se replicam em diversas bibliotecas modernas.

De fato, o `ndarray` também é a base sobre a qual muitas bibliotecas científicas foram construídas. Pacotes como *SciPy*, *Pandas*, *scikit-learn* e *frameworks* de *deep learning* como *PyTorch* e *TensorFlow* utilizam internamente `ndarrays` ou estruturas próprias inspiradas neles para representar tensores multidimensionais. Em *PyTorch*, por exemplo, tensores funcionam de maneira similar ao `ndarray`, mas adicionam suporte a gradientes automáticos e execução em GPU.

Entender o `ndarray` fornece um alicerce sólido para manipular tensores e operações vetorizadas em diferentes contextos. É necessário caminhar na direção de entender os *frameworks* que lidam com tensores e computação de alto desempenho, especialmente importantes na área de inteligência artificial.

Enquanto o `ndarray` foi projetado para maximizar a performance, listas nativas do Python são lentas para operações numéricas. Agora, discutiremos como algumas dessas características (vetorização, *broadcasting* e rotinas otimizadas) são mais rápidas do que a escrita de código baseada em *loops*.

4.1 Evitando *loops* com rotinas otimizadas

O NumPy tem algumas operações matemáticas implementadas de forma muito otimizada. O BLAS é uma das bibliotecas que o NumPy usa para isso. A figura a seguir compara o tempo de execução da multiplicação de matrizes feita manualmente com três *loops* em Python com o tempo da multiplicação feita pelo NumPy. Ele cria matrizes quadradas de vários tamanhos, mede quanto cada método demora, guarda esses valores e depois mostra um gráfico que revela que esse funcionamento é muito mais rápido que o usando *loops* em Python.

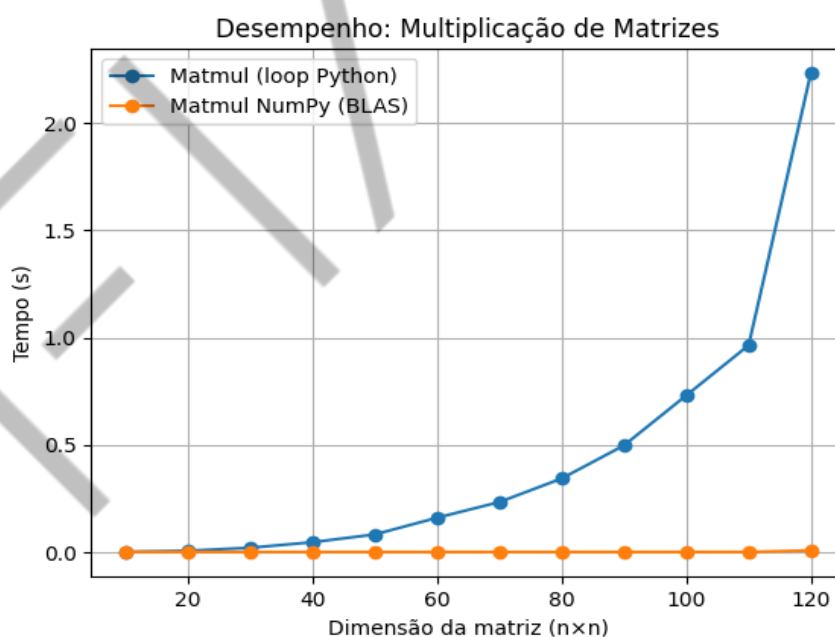


Figura 10 - Desempenho na multiplicação de matrizes com *loops* e com NumPy
Fonte: Elaborada pelo autor (2025)

4.2 Evitando *loops* com vetorização

Suponha que temos um `array` e queremos aplicar uma operação em todos os seus elementos, como realizar uma multiplicação ou calcular uma função matemática

qualquer. A vetorização substitui *loops* por operações de alto nível do NumPy, implementadas em C. Elas são aplicadas automaticamente a todos os elementos do `array` simultaneamente, retornando o novo `array` 1D com os resultados. Isso simplifica o código e torna a execução mais rápida.

A figura “Desempenho da operação x^2 com *loops* e com vetorização” compara quanto tempo leva para calcular o quadrado dos elementos de um `array` de duas formas diferentes: usando um *loop* em Python e usando a operação vetorizada do NumPy através de $y = x*x$. Observamos que a solução vetorizada do NumPy é muito mais rápida do que o *loop* tradicional em Python, especialmente para `arrays` grandes.

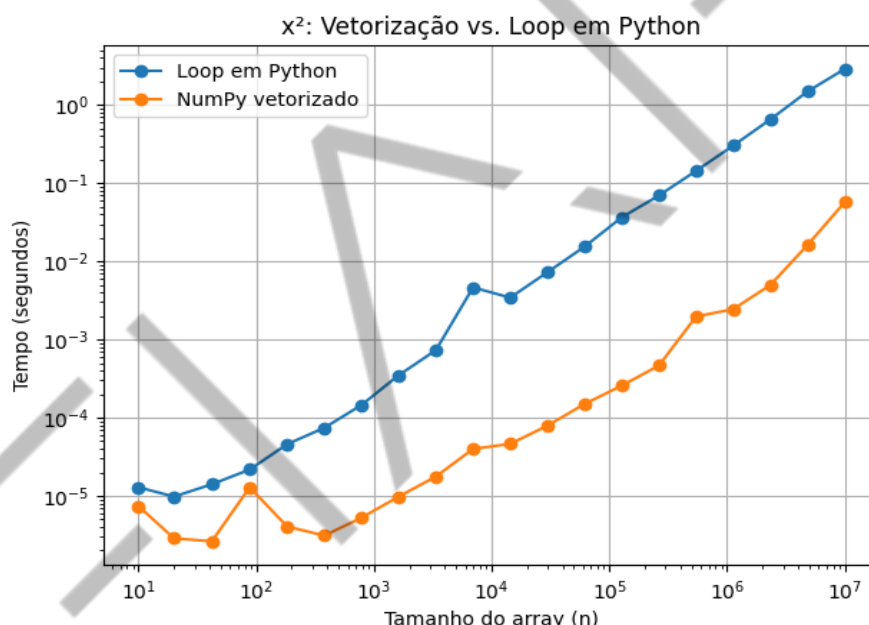


Figura 11 - Desempenho da operação x^2 com *loops* e com vetorização
Fonte: Elaborada pelo autor (2025)

Observação: Python não tem vetorização nativa, só o NumPy. Se a variável x não é um `ndarray` e escrevermos $x = [1, 2, 3]$, o resultado da operação $y = x * 2$ será a repetição da lista $y = [1, 2, 3, 1, 2, 3]$, não uma multiplicação.

4.3 Broadcasting

Broadcasting é um conjunto de mecanismos que o NumPy usa para permitir operações entre `arrays` de tamanhos e formas (*shapes*) diferentes, sem criar cópias

desnecessárias ou expansão manual de `arrays` menores. *Broadcasting* “finge” que um `array` foi expandido, mas sem alocar memória extra.

Observação: o *broadcasting* também é realizado com `ndarrays`. Sem o NumPy, o resultado de `[1, 2, 3] + [1]` resultaria em erro.

O código-fonte “Geração de gráfico 3D utilizando *broadcasting*” e a figura “Gráfico 3D da função $z = \sin(x) + \cos(y)$ ” ilustra esse conceito em um gráfico 3D. Queremos que os valores dos eixos x e y variem de -3 a $+3$ em 50 pontos. Para isso, basta gerar os 50 valores no eixo x e os 50 valores no eixo y que o *broadcasting* já faz as operações para todos os 50×50 pares ordenados.

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(-3, 3, 50)[: , None] # shape (50, 1)
y = np.linspace(-3, 3, 50)[None, :] # shape (1, 50)
z = np.sin(x) + np.cos(y) # broadcasting para (50, 50)
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(x, y, z, cmap='viridis')
plt.show()
```

Código-fonte 7 – Geração de gráfico 3D utilizando *broadcasting*
Fonte: Elaborado pelo autor (2025)

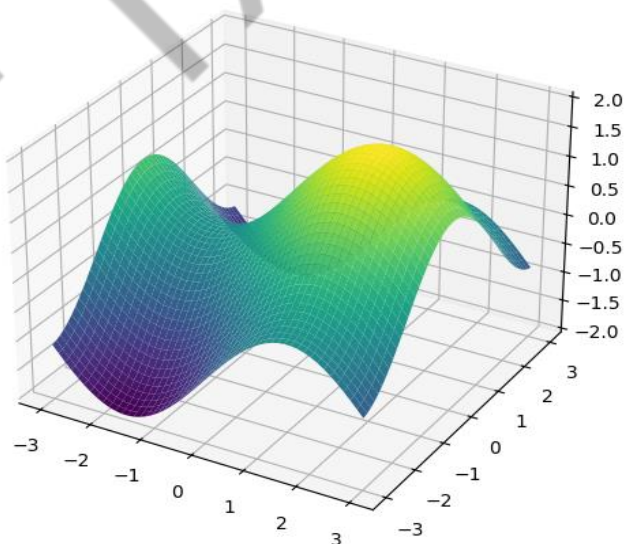


Figura 12 - Gráfico 3D da função $z = \sin(x) + \cos(y)$
Fonte: Elaborada pelo autor (2025)

Observação: a dimensão física que conhecemos não precisa coincidir com a dimensão do `array`. Nesse exemplo, poderíamos armazenar os dados de x , y e z em 3 colunas de um `array` 2D, mesmo representando um gráfico 3D.

4.4 Aproveitando a estrutura de matrizes esparsas

O *SciPy* é uma biblioteca de computação científica que estende o *NumPy*, usando seus *arrays* como base para fornecer funções avançadas de álgebra linear, otimização, estatística e manipulação de matrizes esparsas. Quando uma matriz tem a maioria de seus valores iguais a zero, usar o formato de matriz esparsa do *SciPy* (módulo *sparse*) é mais eficiente do que usar uma matriz cheia.

As figuras “Comparação de tempo da multiplicação $A \times x$ para matrizes cheias e esparsas” e “Comparação do uso da memória de matrizes cheias e esparsas” mostram os resultados de tempo e alocação de memória de uma matriz com cerca de 0,5% de valores não nulos. Vemos que a representação esparsa realiza multiplicações de forma muito mais rápida e usa muito menos memória do que matrizes cheias, à medida que o tamanho das matrizes cresce.

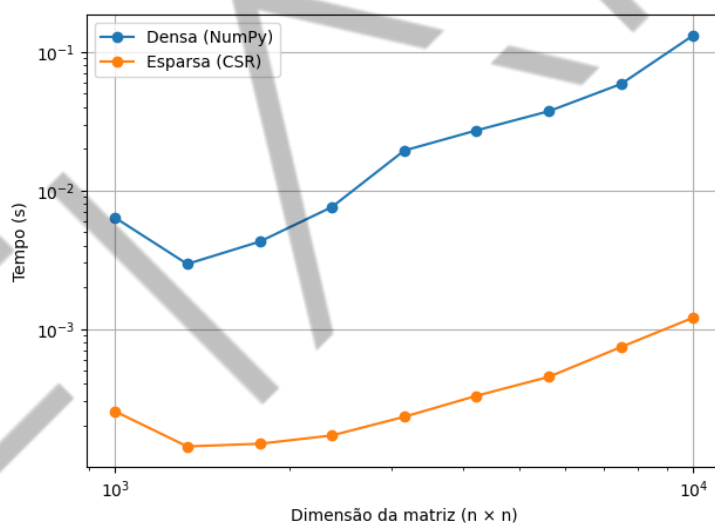


Figura 13 - Comparação de tempo da multiplicação $A \times x$ para matrizes cheias e esparsas
Fonte: Elaborada pelo autor (2025)

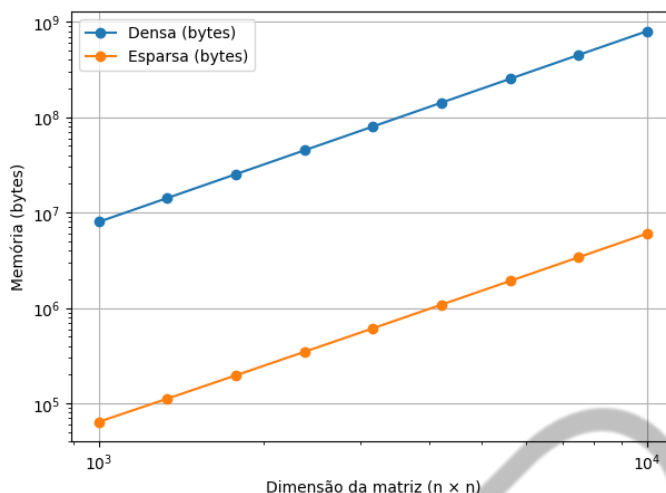


Figura 14 - Comparação do uso da memória de matrizes cheias e esparsas
Fonte: Elaborada pelo autor (2025)

Observação: se a matriz não for esparsa o suficiente, o formato esparsa pode ficar mais lento e ocupar mais memória do que a matriz cheia.

5 APLICAÇÕES

5.1 Sistemas de equações lineares

Um sistema de equações é um conjunto de duas ou mais equações que devem ser resolvidas simultaneamente, pois possuem as mesmas incógnitas. Dois exemplos são mostrados na figura “Sistema de equações sem solução (esquerda) e com solução única (direita)”. O primeiro é quando queremos resolver simultaneamente $y = 0.5x + 2$ e $y = 0.5x - 1$. A imagem (esquerda) mostra que o sistema não tem solução, pois não há nenhum valor de x e y que satisfaça esse sistema. Geometricamente, portanto, as retas das duas expressões não se cruzam. O segundo exemplo é $y = -x + 4$ e $y = 0.7x - 1$. Na direita percebemos que o sistema tem solução aproximadamente em $(2.94, 1.06)$, quando as retas se encontram nesse ponto.

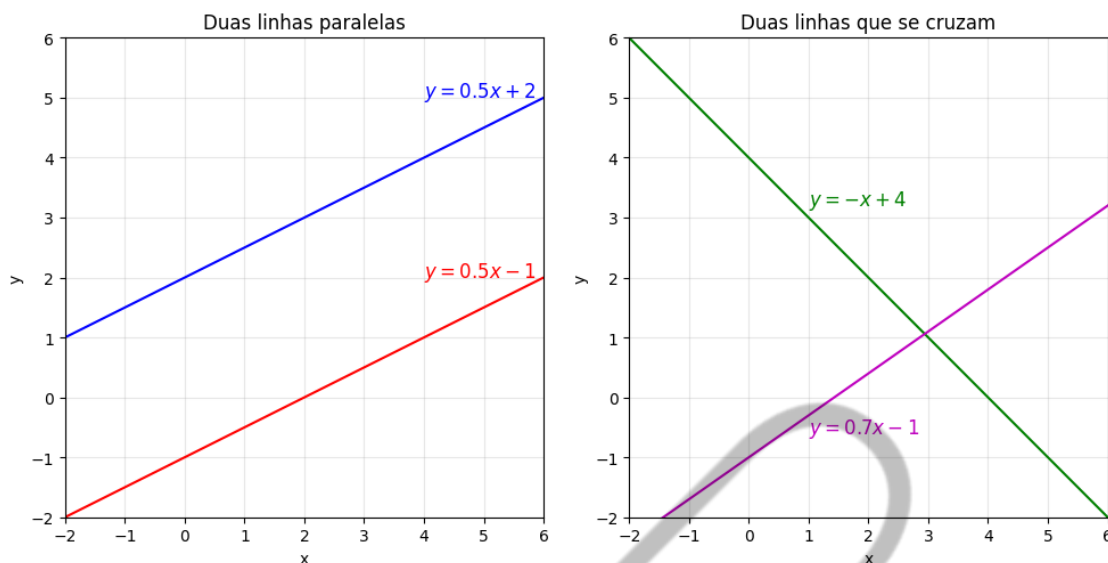


Figura 15 - Sistema de equações sem solução (esquerda) e com solução única (direita)
Fonte: Elaborada pelo autor (2025)

Podemos escrever sistemas lineares na forma matricial $\mathbf{Ax} = \mathbf{y}$, em que $\mathbf{x}$ e $\mathbf{y}$ são vetores e $\mathbf{A}$ é uma matriz. Isso permite associar se o sistema tem soluções com conceitos como matrizes inversas, determinantes e formas escalonadas de matrizes. A figura “Representação matricial de um sistema linear” mostra a notação matemática para essa representação matricial.

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

Figura 16 - Representação matricial de um sistema linear
Fonte: Elaborada pelo autor (2025)

A interpretação do significado dos vetores $\mathbf{x}$ e $\mathbf{y}$ e matriz $\mathbf{A}$ depende do contexto. O vetor $\mathbf{x}$ pode representar incógnitas, intensidades, quantidades ou parâmetros. É essa abstração que permite que esses objetos possam ser manipulados por ferramentas matemáticas semelhantes, independentemente do significado de cada componente.

5.1.1 Exemplo em circuitos elétricos

Um exemplo de solução de sistemas de equações lineares é na análise de circuitos elétricos. Para analisá-los, devemos utilizar as leis de Kirchhoff, descritas a seguir. O foco aqui não é a teoria de circuitos, de modo que essas leis e suas convenções de sinais não serão detalhadas:

- **Lei das correntes de Kirchhoff:** a soma das correntes que chegam a um nó é igual à soma das correntes que saem dele;
- **Lei das tensões de Kirchhoff (LTK):** a soma das quedas e elevações de tensão ao longo de qualquer malha fechada é igual a zero.

De acordo com a figura “Circuito elétrico que ilustra o problema”, vamos considerar que temos uma fonte de 12V, um resistor R1 de 6 ohms no ramo superior e dois ramos em paralelo à direita. O ramo da esquerda possui R2 de 8 ohms em série com R3 de 4 ohms e o ramo da direita possui R4 de 12 ohms. Chamaremos de I1 a corrente que percorre R2 e R3 e de I2 a corrente que percorre R4.

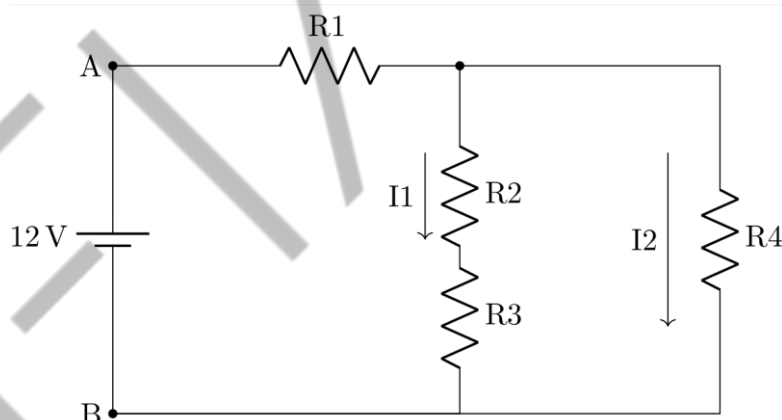


Figura 17 - Circuito elétrico que ilustra o problema
Fonte: Elaborada pelo autor (2025)

A queda de tensão no nodo superior é a mesma para os dois ramos paralelos, de modo que a tensão da fonte menos a queda em R1 deve igualar a queda em cada ramo. Assim, escrevemos:

$$12 - I R_1 = I_1 (R_2 + R_3)$$

$$12 - I R_1 = I_2 R_4$$

Sabendo que a corrente total é $I = I_1 + I_2$, considerando os valores dos resistores e colocando as incógnitas I_1 e I_2 no formato padrão de sistema linear, chegamos a:

$$18 I_1 + 6 I_2 = 12$$

$$6 I_1 + 18 I_2 = 12$$

Esse é o sistema que podemos resolver na forma matricial com NumPy, conforme ilustrado no código-fonte a seguir.

```
import numpy as np
R1, R2, R3, R4 = 6, 8, 4, 12 #Resistores
VS = 12 # fonte
A = np.array([
    [18, 6],
    [6, 18]
], dtype=float)
b = np.array([VS, VS], dtype=float)
I1, I2 = np.linalg.solve(A, b) # função que resolve o sistema linear
print("I1 =", I1)
print("I2 =", I2)
print("I total =", I1 + I2)
```

Código-fonte 8 – Solução de um sistema de equações que representa um circuito elétrico
Fonte: Elaborado pelo autor (2025)

5.2 Processamento de imagens

Uma convolução 2D é um processo em que usamos uma pequena matriz, chamada filtro ou *kernel*, para analisar regiões de uma matriz maior, como uma imagem. O filtro é colocado sobre uma área da imagem e cada valor dele é combinado com o valor correspondente da imagem. Depois essas combinações são resumidas em um único número, que indica o quanto aquela região se parece com o padrão representado pelo filtro. Em seguida, o filtro é deslocado para outra posição e o procedimento se repete até cobrir toda a imagem.

Vamos ilustrar a convolução de uma imagem com um filtro laplaciano na figura “Imagem original (esquerda) e bordas destacadas (direita)” e no código-fonte “Definição de filtro e aplicação de convolução 2D para detecção de bordas”. O resultado é uma nova imagem que indica, em cada ponto, o quanto aquela região contém uma mudança brusca de intensidade. Isso é útil para destacar bordas, já que

os valores ficam altos onde a imagem apresenta contornos marcados e baixos onde a variação é suave.

Destaca-se que a convolução é a base das redes neurais convolucionais, que buscam os filtros ótimos para representar as características que melhor explicam as imagens para uma determinada tarefa, como classificação ou detecção de objetos.

```
import numpy as np
from scipy.signal import convolve2d
from skimage import color, io
img = io.imread("image.jpg") # Carregar imagem
gray = color.rgb2gray(img) # Converter para tons de cinza

# Filtro Laplaciano para detecção de bordas
laplacian = np.array([
    [0, -1, 0],
    [-1, 4, -1],
    [0, -1, 0]
])
# Aplicar convolução 2D
edges = convolve2d(gray, laplacian, mode='same', boundary='symm')
```

Código-fonte 9 – Definição de filtro e aplicação de convolução 2D para detecção de bordas
Fonte: Elaborado pelo autor (2025)

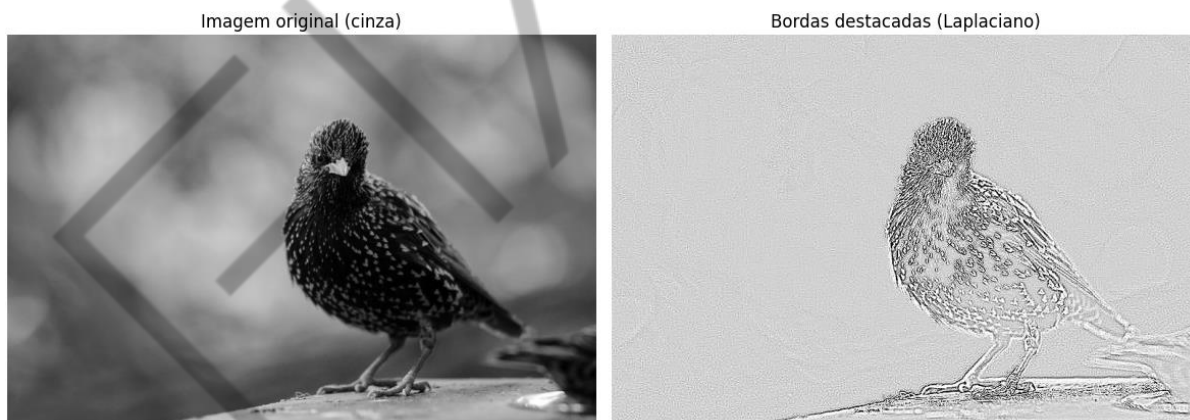


Figura 18 - Imagem original (esquerda) e bordas destacadas (direita)
Fonte: Elaborada pelo autor (2025)

5.3 Ajuste de curvas

O ajuste de curvas visa encontrar uma expressão matemática que modele a relação entre variáveis independentes e dependentes. No caso 2D, temos uma variável dependente y e uma variável independente x . Com isso, busca-se calcular os valores em outros pontos além dos dados coletados originalmente.

Nesse sentido, algoritmos de otimização buscam os melhores parâmetros do modelo que levam à curva de "melhor ajuste", isto é, que minimize o erro total entre a curva e os pontos observados. O método mais comum para determinar essa curva é o dos mínimos quadrados, que busca minimizar a soma dos erros quadráticos entre os dados coletados e a saída do modelo. Isso envolve a definição do sistema linear que representa o problema e operações entre matrizes e vetores para minimizar as diferenças.

No exemplo do código-fonte “Ajustes linear e polinomial”, a figura “Resultado dos ajustes linear e polinomial” mostra o resultado de um ajuste de linha reta (polinômio de grau 1) e de uma parábola (polinômio de grau 2) aos dados experimentais. Para isso, foi utilizada a função `np.polyfit(x, y, deg)`, que calcula os coeficientes da curva de melhor ajuste. Os coeficientes resultantes (`coef_lin` e `coef_poly`) são então utilizados para calcular novos valores da curva de ajuste em outros pontos.

```
import numpy as np
import matplotlib.pyplot as plt
# Pontos amostrais
x = np.array([0, 1, 2], dtype=float)
y = np.array([1, 2, 5], dtype=float)
# --- Regressão linear (grau 1) ---
coef_lin = np.polyfit(x, y, deg=1)
def p_lin(x_val):
    return coef_lin[0] * x_val + coef_lin[1]
# --- Regressão polinomial (grau 2)
coef_poly = np.polyfit(x, y, deg=2)
def p_poly(x_val):
    return coef_poly[0] * x_val**2 + coef_poly[1] * x_val + coef_poly[2]
# --- Plot ---
x_plot = np.linspace(min(x)-0.5, max(x)+0.5, 200)
y_lin = p_lin(x_plot)
y_poly = p_poly(x_plot)
```

Código-fonte 10 – Ajustes linear e polinomial

Fonte: Elaborado pelo autor (2025)

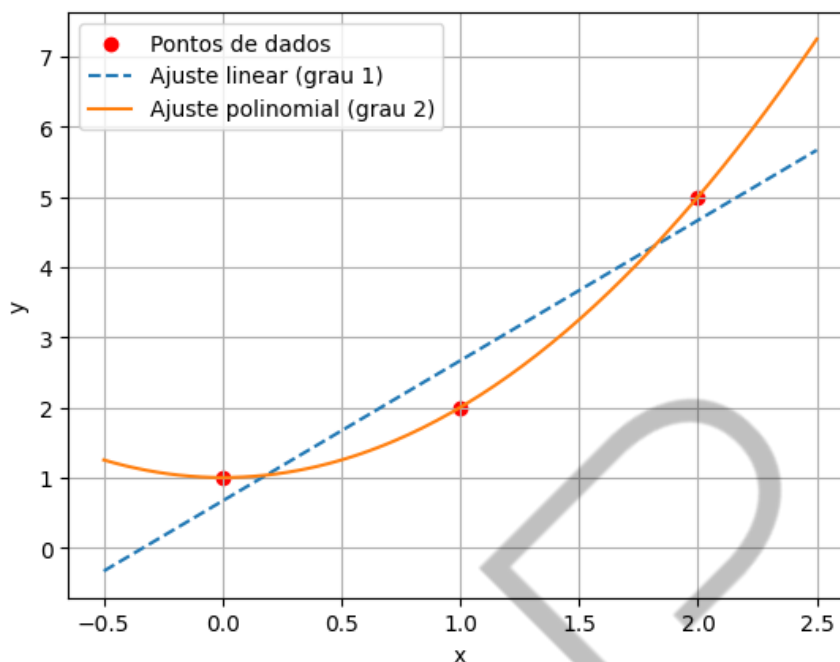


Figura 19 - Resultado dos ajustes linear e polinomial
Fonte: Elaborado pelo autor (2025)

5.4 PCA para redução de dimensionalidade

Imagine que você tem uma grande planilha com dezenas de colunas sobre clientes de uma loja online: idade, renda, gastos, frequência de compra, tempo no site, respostas a pesquisas e assim por diante. Cada coluna descreve uma característica (*feature*). O número de *features* é o número de dimensões que o nosso problema possui. Além disso, quanto mais dimensões temos, mais difícil fica visualizar e analisar os dados. Assim, técnicas de redução de dimensionalidade visam resumir esse conjunto enorme de informações usando menos dimensões, mas ainda mantendo o essencial dos dados.

Uma técnica muito usada é a Análise de Componentes Principais (do inglês *Principal Component Analysis* - PCA). Mas, antes de entender o PCA, é importante diferenciar dois conceitos:

- *Feature selection* (seleção de *features*): escolher algumas colunas originais e descartar outras. As colunas mantidas continuam com o mesmo significado;
- Redução de dimensionalidade (como o PCA): criar novas variáveis que combinam várias colunas ao mesmo tempo. Cada uma é chamada de componente principal. Desse modo, menos informação disponível é

perdida, mas, nessa combinação, não se tem mais o significado das colunas originais.

As componentes principais capturam o padrão geral desse grupo de variáveis, encontrando o conjunto de componentes que melhor representa a variação dos dados. Logo, o PCA é ótimo para compressão dos dados, visualização e aceleração de algoritmos.

Por trás desse processo, existe uma sequência de operações matemáticas. Tudo começa observando como as variáveis mudam juntas. Se duas colunas aumentam e diminuem ao mesmo tempo, elas podem estar correlacionadas. Muitas vezes, várias delas dizem quase a mesma coisa: “gastos mensais”, “número de compras” e “valor médio por compra”, o que indica informação redundante. Para medir essas relações, calcula-se a matriz de covariância dos dados.

Em seguida, o PCA procura direções especiais no espaço dos dados, chamadas autovetores, que indicam os eixos onde os dados variam mais. Cada um desses eixos possui um autovalor, que diz quanta variação aquela direção explica. Eles são ordenados da direção mais informativa para a menos informativa.

Ao final, projeta-se os dados nessas novas direções. Se mantivermos apenas os primeiros componentes principais, o objetivo é conseguir uma representação mais compacta que preserva aquilo que mais importa nos dados.

Uma forma de visualizar o procedimento do PCA é mostrada na figura “Ilustração de duas componentes principais de um conjunto de dados”. Primeiro, essa nuvem é centralizada e analisada para saber em quais direções ela se espalha mais. Depois, o espaço é girado de modo que os novos eixos (setas vermelhas) fiquem alinhados com essas direções de maior variação.

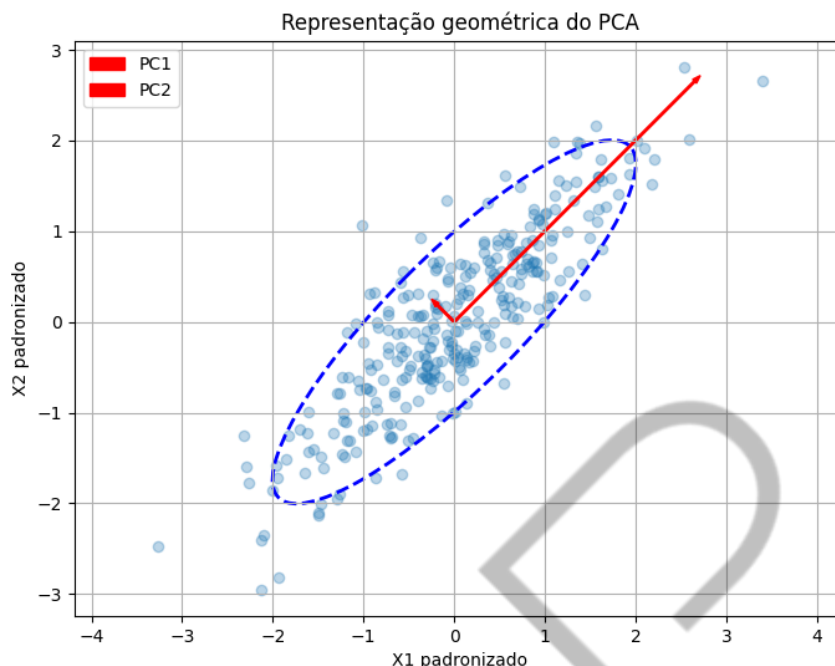


Figura 20 - Ilustração de duas componentes principais de um conjunto de dados
Fonte: Elaborada pelo autor (2025)

5.4.1 Exemplo de uso de PCA em classificação

No exemplo a seguir, utilizaremos um *dataset* real, que descreve características químicas de vinhos de 3 origens cultivadas diferentes. São 13 características no total, incluindo álcool, ácido málico, cinzas, alcalinidade das cinzas, magnésio, fenóis totais e flavonoides. O código-fonte “PCA do *wine dataset*” realiza as operações do PCA.

A figura “Visualização das duas primeiras componentes principais no *wine dataset*, com e sem *standardization* dos dados” mostra que apenas 2 componentes do PCA já mantêm grande parte da informação que antes estava espalhada em 13 *features*. Um algoritmo de classificação poderia ser utilizado diretamente nesse espaço reduzido. Por outro lado, a imagem também mostra que, sem padronização dos dados, o PCA não é tão eficiente e os dados de classes diferentes ficam sobrepostos.

```
from sklearn.datasets import load_wine
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
# Carrega dados
X, y = load_wine(return_X_y=True)
# PCA sem padronização
X_pca = PCA(n_components=2).fit_transform(X)
# PCA com padronização
```

```
X_std = StandardScaler().fit_transform(X)
X_pca_std = PCA(n_components=2).fit_transform(X_std)
```

Código-fonte 11 – PCA do *wine dataset*

Fonte: Elaborado pelo autor (2025)

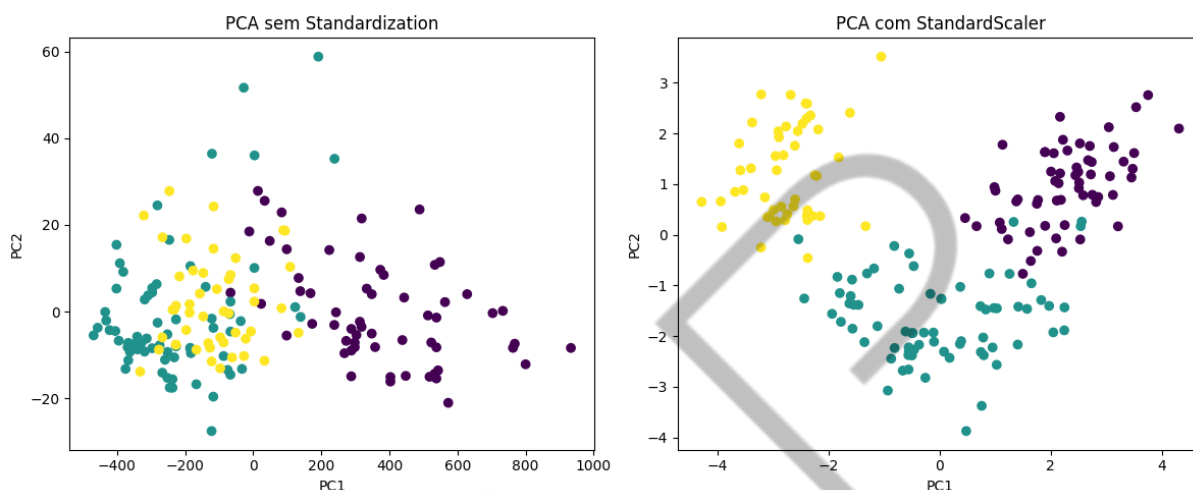


Figura 21 - Visualização das duas primeiras componentes principais no *wine dataset*, com e sem *standardization* dos dados

Fonte: Elaborada pelo autor (2025)

Observação: quando aplicamos PCA, não é obrigatório usar apenas 2 componentes. Na prática, muitas vezes usamos 5, 10 ou 20 componentes, dependendo do quanto de informação queremos preservar. No entanto, 2 dimensões permitem um gráfico 2D visualmente intuitivo, sendo a forma mais simples possível de mostrar como os dados ficam “espalhados” após a redução de dimensionalidade.

6 CONCLUSÃO

Ao longo deste capítulo, vimos como vetores e matrizes são elementos centrais na relação entre álgebra linear e computação. A representação desses objetos matemáticos em linhas e colunas, indexadas por números inteiros, é inerentemente discreta, o que os torna particularmente adequados para implementação em computadores digitais. Ela permite que operações matemáticas sejam formuladas através de algoritmos e executadas de forma sistemática.

Em Python, matrizes e vetores podem ser representados por `arrays`, como os fornecidos pela biblioteca `NumPy`. Embora exista uma correspondência direta entre muitos conceitos da álgebra linear e estruturas de dados como `arrays` multidimensionais, eles não são a mesma coisa. Vetores e matrizes são objetos

matemáticos abstratos, definidos por propriedades e relações, enquanto `arrays` são estruturas de dados concretas manipuláveis por programas, organizadas na memória e sujeitas a limitações de armazenamento e desempenho.

Em aplicações práticas, a eficiência computacional deve ser avaliada. A forma como as operações são expressas impacta diretamente o tempo de execução e o uso de recursos. Operações matematicamente simples podem se tornar inviáveis se implementadas de forma ingênua. Nesse sentido, destacamos estratégias para evitar *loops* explícitos, explorar vetorização, *broadcasting* e representação esparsa, que podem diminuir o tempo de execução e custo computacional quando queremos executar operações sobre grandes volumes de dados.

Por fim, apresentamos um conjunto de aplicações com o objetivo de contextualizar os conceitos discutidos. Exemplos envolvendo sistemas de equações lineares, processamento de sinais e imagens, ajuste de curvas e redução de dimensionalidade ilustram a abrangência do uso de vetores e matrizes em diferentes áreas. Esses exemplos servem como ponto de partida para estudos mais aprofundados na solução de problemas reais nas áreas de ciência de dados, engenharia e inteligência artificial.

REFERÊNCIAS

BRASIL, Reyolando Manoel Lopes Rebello da Fonseca; BALTHAZAR, José Manoel; GÓIS, Wesley. **Métodos numéricos e computacionais na prática de engenharias e ciências**. São Paulo, SP: Blücher, 2015. 185 p., il. ISBN 9788521209348.

BURDEN, Richard L.; FAIRES, J. Douglas; BURDEN, Annette M. **Análise numérica**. Tradução de Helena Maria Ávila de Castro; Tradução de All Tasks. 3. ed. São Paulo, SP: Cengage Learning, c2016. xvi, 879 p., il. ISBN 9788522123407.

FRANCO, Neide Bertoldi. **Álgebra linear**. São Paulo, SP: Pearson, 2016. ix, 362 p., il. ISBN 9788543019154.

FRANCO, Neide Bertoldi. **Cálculo numérico**. São Paulo, SP: Pearson, 2006. xii, 505 p., il. ISBN 9788576050872.

GOLUB, Gene H.; VAN, Charles L. **Matrix computations**. 4. ed. Baltimore, USA: Johns Hopkins University Press, c1996. xvii, 756 p., il. ISBN 9781421407944.

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. **Deep learning**. Cambridge, USA: MIT Press, 2017. xi, 775 p., il. (Adaptive computation and machine learning). ISBN 9780262035613.

LEON, Steven J. **Álgebra linear com aplicações**. Tradução de Sérgio Gilberto Taboada. 8. ed. Rio de Janeiro, RJ: LTC, c2011. xi, 451 p., il. ISBN 9788521617693.

MUELLER, Jennifer L.; SILTANEN, Samuli. **Linear and nonlinear inverse problems with practical applications**. Philadelphia, USA: Society for Industrial and Applied Mathematics, 2012. xiii, 351 p., il. (Computational science and engineering series). ISBN 9781611972337.

GLOSSÁRIO

Algoritmo	Conjunto finito de instruções bem definidas que descrevem, de modo sistemático, como resolver um problema ou executar uma tarefa.
Álgebra linear	Ramo da matemática que estuda vetores, matrizes, transformações lineares e sistemas de equações lineares, concentrando-se em estruturas que permitem soma e multiplicação por escalares.
Axioma	Um axioma é uma afirmação básica da matemática que aceitamos como verdadeira sem precisar provar. Ele serve como ponto de partida para construir regras, fórmulas e teoremas.
Dados	Valores registrados que representam alguma informação. Podem ser números, textos, sinais, medições ou qualquer forma de registro que possa ser armazenado, transmitido ou processado. São elementos brutos, ainda sem interpretação.
Direção (vetor)	Característica que indica para onde o vetor aponta, independentemente de sua magnitude.
Equação linear	Equação em que as incógnitas aparecem de forma simples (grau 1), apenas multiplicadas por constantes e somadas. Não há produtos entre incógnitas ou termos não lineares como potências maiores do que 1.

Escalar	Valor numérico isolado, que pode representar uma magnitude ou atuar como fator multiplicativo em operações lineares.
Espaço vetorial	Conjunto de objetos (vetores) em que estão definidas operações de soma e multiplicação por escalares, obedecendo a axiomas específicos.
Magnitude (vetor)	Medida do tamanho do vetor.
Matriz	Arranjo retangular de números organizado em linhas e colunas. Entre outras coisas, pode representar transformações lineares e sistemas de equações lineares.
Sentido (vetor)	Indica qual das duas direções opostas, ao longo de uma mesma linha, o vetor aponta.
Sistema de equações lineares	Conjunto de equações lineares que compartilham as mesmas incógnitas e devem ser resolvidas simultaneamente.
Tensor	Objeto matemático que generaliza conceitos de escalares, vetores e matrizes para dimensionalidades maiores.
Vetor	Elemento de um espaço vetorial. De modo específico, é um objeto matemático que não pode ser representado apenas por um escalar. Em contextos geométricos, é representado por uma entidade com magnitude, direção e sentido. Na física, costuma ser ilustrado através de setas.